27. Breadth First Search (BFS)

AIM

To write a Prolog program to implement **Breadth First Search (BFS)** for traversing a graph.

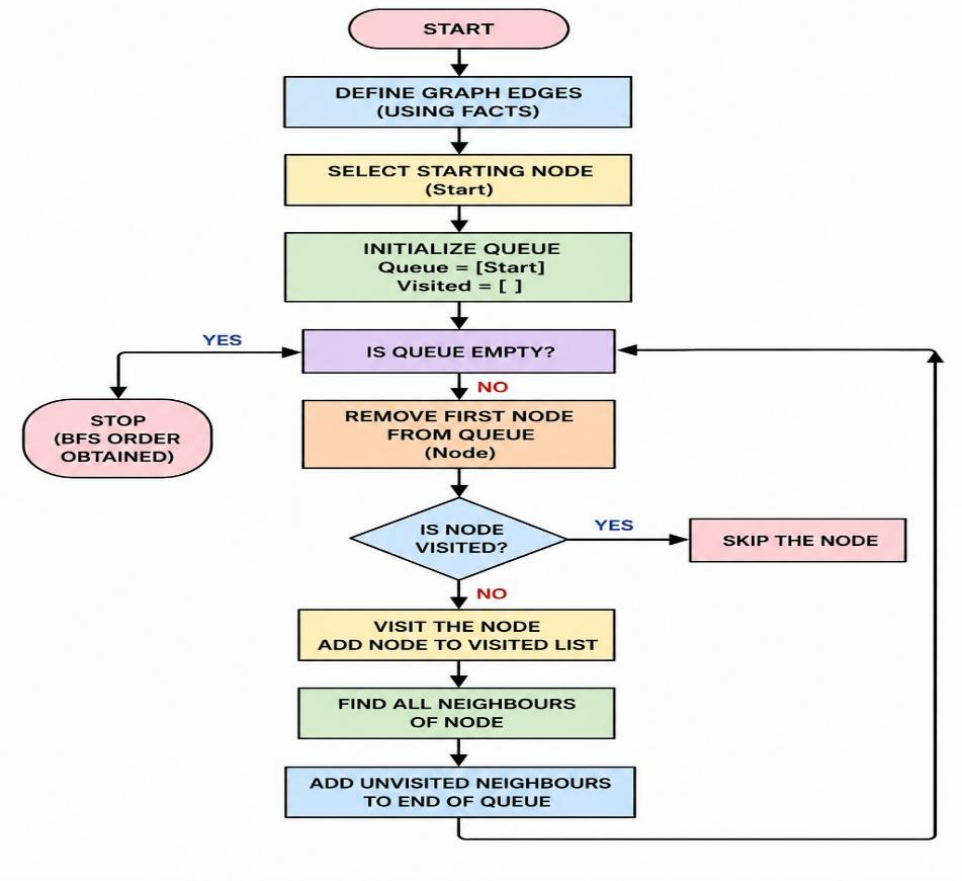
OBJECTIVE

- To understand the concept of Breadth First Search.
- To implement BFS using Prolog.
- To traverse a graph level by level.
- To understand the use of lists and recursion in Prolog.

ALGORITHM

1. Start the program.
2. Define the edges of the graph using facts.
3. Represent the graph as connected nodes.
4. Initialize the queue with the starting node.
5. Remove the first node from the queue.
6. Visit the node if it has not already been visited.
7. Add its unvisited neighboring nodes to the end of the queue.
8. Continue until the queue becomes empty.
9. Display the BFS traversal order.
10. Stop.

FLOWCHART

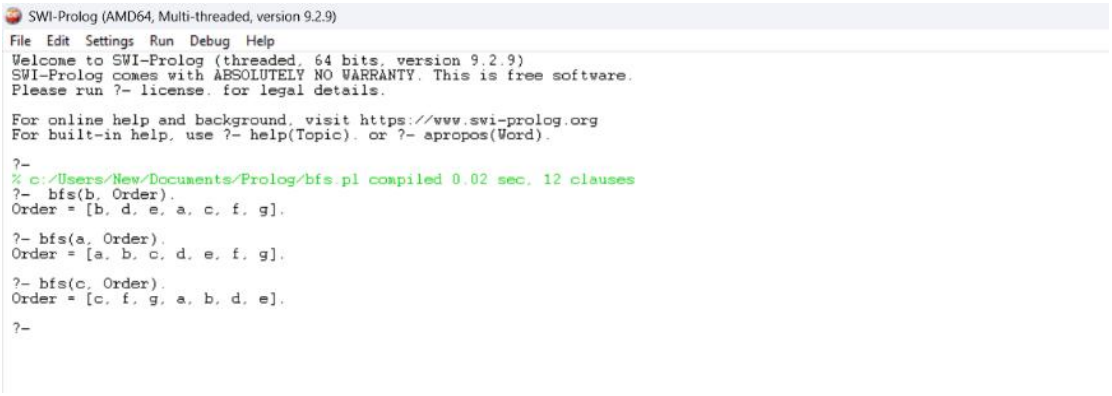


CODE:

```
% GRAPH EDGES
edge(a, b).
edge(a, c).
edge(b, d).
edge(b, e).
edge(c, f).
edge(c, g).
% MAKE THE GRAPH BIDIRECTIONAL
connected(X, Y) :-
    edge(X, Y).
connected(X, Y) :-
```

```
    edge(Y, X).
% BFS IMPLEMENTATION
bfs(Start, Order) :-
    bfs_queue([Start], [], Order).
bfs_queue([], _, []).
bfs_queue([Node|Queue], Visited, Order) :-
    member(Node, Visited),
    bfs_queue(Queue, Visited, Order)
bfs_queue([Node|Queue], Visited, [Node|Order]) :-
    \+ member(Node, Visited),
    findall(Next,
            (connected(Node, Next),
             \+ member(Next, Visited),
             \+ member(Next, Queue)),
            Neighbours),
    append(Queue, Neighbours, NewQueue),
    bfs_queue(NewQueue, [Node|Visited], Order).
```

OUTPUT:



RESULT:

Thus, the Breadth First Search (BFS) algorithm was successfully implemented in Prolog. The graph was traversed level by level, starting from the specified node.